

SPRING SECURITY SHORT NOTES

1. Authentication

- Authentication means verifying user identity.
- Example: username/password login.
- Answer to: "Who are you?"

2. Authorization

- Authorization means checking permissions/access.
- Example: ADMIN can delete users.
- Answer to: "What can you access?"

3. Session Authentication

- After successful login, server creates HttpSession.
- Tomcat generates JSESSIONID automatically.
- Browser stores JSESSIONID cookie.
- Browser sends cookie automatically with future requests.

4. Session Storage

- Server stores user/session data.
- Browser stores only session ID (JSESSIONID).

5. Why Browser Sends Session Automatically

- Server sends:
Set-Cookie: JSESSIONID=...
- Browser automatically stores and sends it.

6. CSRF (Cross Site Request Forgery)

- Attack where malicious website sends fake request using user's valid session.

7. Why Session ID Alone is Not Enough

- Browser automatically sends JSESSIONID even for malicious requests.
- Session proves user is logged in.
- Session does NOT prove request came from trusted frontend.

8. CSRF Token

- Extra secret token used to validate genuine request.
- Spring compares:
Session CSRF Token == Request CSRF Token
- If mismatch → 403 Forbidden.

9. CSRF Token Storage

- Usually stored in server session.
- Frontend manually sends it in request header/form.
- Browser does NOT auto-send CSRF token.

10. How to Get CSRF Token

- Spring stores token in request attribute "_csrf".

Example:

```
CsrfToken csrf =  
(CsrfToken) request.getAttribute("_csrf");
```

Correct Import:

```
import org.springframework.security.web.csrf.CsrfToken;
```

11. CSRF Header Name

- Usually:
X-CSRF-TOKEN

12. Why GET Works Without CSRF

- GET is treated as safe/read-only request.
- CSRF protection mainly applies to:
POST, PUT, DELETE

13. JWT (JSON Web Token)

- JWT is used for stateless authentication.
- Contains user data inside token itself.
- Example:
username, role, expiry

14. JWT Flow

Login
↓
Server creates JWT
↓
Client stores JWT
↓
Every request sends JWT

15. Why JWT Usually Doesn't Need CSRF
- Browser does NOT automatically send JWT.
 - Frontend manually sends:
Authorization: Bearer token
 - Fake websites cannot easily send it.

16. Session vs JWT

SESSION:

- Stateful
- Server stores user data
- Uses JSESSIONID
- CSRF protection needed

JWT:

- Stateless
- Server stores nothing
- Token contains user data
- Usually no CSRF needed

17. Spring Security Dependency

Wrong:
spring-security-core

Correct:
spring-boot-starter-security

18. Important Spring Security Classes

SecurityFilterChain
- Main security configuration

AuthenticationManager
- Validates user credentials

UserDetailsService
- Loads user from DB

PasswordEncoder
- Encrypts password

SecurityContextHolder
- Stores logged-in user details

OncePerRequestFilter
- Custom JWT filter

19. Security Filter Flow

Request
↓
Security Filters
↓
Authentication
↓
Authorization
↓
Controller

20. Disable CSRF For REST APIs

.csrf(csrf -> csrf.disable())
Common when using JWT authentication.

21. Important One-Line Differences

Authentication:

- Who are you?

Authorization:

- What can you access?

Session:

- Server-side login state

CSRF:

- Prevent fake requests

JWT:

- Stateless authentication token